



Universidad de Guayaquil
Facultad de Ingeniería Industrial
Carrera de Ingeniería en Telemática

Materia:

Ingeniería de Software

Tema:

Etapas 2: Análisis de las Inteligencias Artificiales

Grupo de Trabajo:

Calero Matos Anthony David

Intriago Celi Bladimir Sebastian

Rodriguez Calderon Jordan Josue

Vicuña Aspiazu Mario Samuel

Docente:

Ing. Ángel Plaza Vargas, MSC.

Fecha de Entrega:

28 de mayo de 2026

Año Lectivo:

2025 – 2026 CI

Contenido

1. Introducción.....	4
2. Objetivos.....	4
2.1. Objetivo General.....	4
2.2. Objetivos Específicos.....	4
3. Metodología de Comparación.....	4
3.1. Inteligencias Artificiales Seleccionadas.....	4
3.2. Condiciones de Evaluación.....	5
3.3. Tecnologías del Proyecto.....	5
4. Prompt 1.....	5
5. Respuesta de Gemini.....	6
5.1. Descripción General.....	6
5.2. Arquitectura Propuesta.....	6
5.3. Organización de Carpetas.....	6
5.4. Diseño de Clases.....	7
5.5. Ventajas Observadas.....	7
5.6. Desventajas Observadas.....	8
6. Respuesta de DeepSeek.....	8
6.1. Descripción General.....	8
6.2. Arquitectura Propuesta.....	8
6.3. Organización de Carpetas.....	8
6.4. Diseño de Clases.....	10
6.5. Ventajas Observadas.....	10
6.6. Desventajas Observadas.....	10
7. Respuesta de Claude.....	11
7.1. Descripción General.....	11

7.2. Arquitectura Propuesta.....	11
7.3. Organización de Carpetas.....	11
7.4. Diseño de Clases.....	14
7.5. Ventajas Observadas.....	15
7.6. Desventajas Observadas.....	15
8. Comparación Técnica Inicial.....	15
8.1. Criterios de Evaluación.....	15
8.2. Escala de Valoración.....	15
8.3. Tabla Comparativa.....	16
8.4. Gráfico Comparativo.....	16
9. Análisis de Resultados.....	17
9.1. Organización.....	17
9.2. Escalabilidad.....	17
9.3. Claridad.....	18
9.4. Calidad General.....	18
9.5. Observaciones.....	18
10. Discusión.....	18
11. Resultados Obtenidos.....	19
12. Conclusiones.....	19
Anexo.....	20
Referencias.....	20

1. Introducción

La creciente incorporación de herramientas de inteligencia artificial en el desarrollo de software ha generado nuevas posibilidades para automatizar tareas relacionadas con el diseño, programación y documentación de sistemas informáticos. Actualmente existen diversas plataformas de IA generativa capaces de asistir a los desarrolladores mediante la generación de propuestas de código, arquitecturas de software, documentación técnica y soluciones a problemas específicos.

En esta etapa se realizará un análisis comparativo entre tres inteligencias artificiales: Gemini, Claude y DeepSeek, con el objetivo de evaluar su capacidad para participar en el diseño inicial del videojuego "El Último Bastión". Para garantizar una comparación justa, las tres IA recibirán exactamente los mismos requerimientos, restricciones y condiciones de desarrollo mediante un Prompt Maestro previamente definido por el equipo.

2. Objetivos

2.1. Objetivo General

Analizar y comparar las propuestas de arquitectura de software generadas por diferentes inteligencias artificiales para el desarrollo del videojuego "El Último Bastión".

2.2. Objetivos Específicos

- Diseñar un Prompt Maestro común para todas las IA.
- Obtener propuestas de arquitectura de software.
- Analizar la organización de cada propuesta.
- Comparar la calidad de las respuestas generadas.
- Identificar ventajas y limitaciones de cada IA.

3. Metodología de Comparación

3.1. Inteligencias Artificiales Seleccionadas

Las herramientas seleccionadas fueron:

IA	Función
Gemini	Generación de propuestas
Claude	Generación de propuestas
DeepSeek	Generación de propuestas

3.2. Condiciones de Evaluación

Para asegurar imparcialidad:

- Se utilizó el mismo prompt.
- Se utilizaron los mismos requerimientos.
- Se emplearon las mismas tecnologías.
- Se solicitaron los mismos entregables.
- No se realizaron modificaciones posteriores.

3.3. Tecnologías del Proyecto

- Python 3
- Pygame
- GitHub
- Visual Studio Code

4. Prompt 1

Objetivo: Definir un único conjunto de instrucciones para todas las IA participantes.

Verificar Archivo: Anexo 1 Prompt 1 Arquitectura

https://github.com/mariovicunaa/Proyecto_El_Ultimo_Bastion/blob/main/Ep_2_Analisis_I

A/Anexo_1_Prompt_1_Arquitectura.pdf

El uso de un Prompt garantiza que las diferencias observadas en los resultados sean consecuencia de las capacidades de cada inteligencia artificial y no de variaciones en las instrucciones proporcionadas.

5. Respuesta de Gemini

5.1. Descripción General

Gemini propone una Arquitectura Basada en Componentes y Gestores (Manager Pattern) operando bajo un Game Loop clásico y gobernado por una Máquina de Estados Finitos (FSM). Enfatiza el bajo acoplamiento utilizando un Patrón Observador (Event Bus) para la comunicación entre sistemas.

5.2. Arquitectura Propuesta

El sistema consta de un motor de juego con un bucle constante a 60 FPS que procesa entradas, actualiza la lógica y renderiza. Utiliza un gestor de estados para controlar la interfaz activa (Menú, Jugando, Pausa, Derrota) y carga inicial de assets.

5.3. Organización de Carpetas

el_ultimo_bastion/

— assets/	# Recursos multimedia
— images/	# Sprites de pixel art/cartoon (torres, enemigos, UI)
— audio/	# Música de fondo y efectos de sonido (SFX)
— fonts/	# Fuentes tipográficas
— maps/	# Archivos de datos del mapa (ej. Tiled, JSON)
— config/	# Archivos de configuración general
— settings.json	# Resoluciones, volumen, atajos de teclado
— game_data.json	# Estadísticas de enemigos, costos de torres, vida del bastión
— docs/	# Documentación técnica y diagramas UML

— src/	# Código fuente principal
— core/	# Bucle principal, gestor de estados
— entities/	# Clases base, enemigos, torres, proyectiles
— managers/	# Gestores lógicos (oleadas, recursos, construcción)
— ui/	# Elementos de interfaz gráfica (botones, HUD)
— utils/	# Funciones auxiliares, matemáticas, colisiones
— world/	# Lógica del mapa, grilla de construcción, pathfinding
— tests/	# Pruebas unitarias y de integración
— .gitignore	# Archivos a ignorar por Git (ej. __pycache__)
— requirements.txt	# Dependencias del proyecto (ej. pygame-ce==2.3.2)
— main.py	# Punto de entrada de la aplicación

5.4. Diseño de Clases

Las clases principales se basan en herencia y composición:

GameManager: Orquesta la partida global.

Entity: Clase base gráfica y física, de la cual heredan Enemy, Tower y Projectile.

WaveManager: Gestiona la aparición de enemigos.

Bastion: Maneja la vitalidad del núcleo.

Grid/Map: Maneja la lógica espacial y pathfinding.

5.5. Ventajas Observadas

Diseño Basado en Datos (Data-Driven): Los atributos no están hardcodeados en código, sino en archivos JSON.

Patrón Factory y Event Bus: Permite instanciar objetos fácilmente y mantener los componentes desacoplados.

Recomendaciones académicas: Sugiere modelado UML, validación de variables con Type Hinting en Python y entornos virtuales.

5.6. Desventajas Observadas

Advierte que calcular distancias y colisiones por frame requiere optimizaciones severas como el Object Pooling y división espacial, si no impactará el rendimiento.

Existe un riesgo de crear código espagueti en la UI si no se implementa el modelo MVC estrictamente.

6. Respuesta de DeepSeek

6.1. Descripción General

DeepSeek apuesta por una arquitectura sumamente estructurada usando capas clásicas y una sólida máquina de estados para el flujo. Destaca por su alta cohesión y diseño completamente orientado a objetos mediante referencias directas.

6.2. Arquitectura Propuesta

Consta de un motor principal (Game Loop) que procesa la lógica a través del estado de juego activo (GameplayState). Este estado aloja a los diferentes "Managers" (Wave, Resource, Collision, etc.) interactuando centralizadamente.

6.3. Organización de Carpetas

el-ultimo-bastion/

```

├── main.py          # Punto de entrada, inicializa Pygame y arranca el juego
├── config.py        # Constantes globales (resolución, FPS, rutas, balances)
├── src/
│   ├── core/
│   │   ├── game.py    # Clase principal Game (bucle, estados)
│   │   ├── states.py  # Estados de la máquina (MenuState, GameplayState, etc.)
│   │   └── entities/
│   │       ├── enemy.py    # Clase base Enemy y subclases (Saqueador, Arquero, Forzudo)
│   │       └── defense.py  # Clase base Defense y subclases (TorreArquera, Muro, Balista)

```



```

| | | — projectile.py    # proyectiles disparados por defensas
| | | — bastion.py      # El bastión (vida, posición)
| | — managers/
| | | — wave_manager.py  # Control de oleadas, generación de enemigos
| | | — resource_manager.py # Gestión de oro y maná, validación de gastos
| | | — input_manager.py # Procesamiento de eventos de teclado y ratón
| | | — collision_manager.py # Lógica de colisiones entre proyectiles y enemigos
| | — map/
| | | — grid.py         # Cuadrícula para construir defensas
| | | — path.py         # Definición del camino (waypoints)
| | | — tile.py         # Representación de cada celda del mapa
| | — ui/
| | | — button.py       # Botón genérico para menús
| | | — hud.py          # Interfaz durante el juego (vida, recursos, oleada)
| | | — menu.py         # Pantalla de menú principal
| | | — pause_menu.py   # Pantalla de pausa
| | | — game_over.py    # Pantalla de derrota
| | — utils/
| | | — asset_loader.py  # Carga de imágenes, sonidos, fuentes
| | | — timer.py        # Temporizador para cooldowns y oleadas
| | | — vector.py       # Clase Vector2D para cálculos matemáticos
| — assets/
| | — graphics/
| | | — enemies/       # Sprites de goblins
| | | — defenses/      # Sprites de torres, muros, balista
| | | — tiles/         # Imágenes del terreno

```

ui/	# Botones, fondos de menú
effects/	# Projectiles, explosiones (futuro)
sounds/	
music/	
sfx/	
fonts/	# Fuentes bitmap
docs/	# Documentación técnica y de diseño
tests/	# Pruebas unitarias (p.ej., con pytest)
data/	# Archivos de definición de niveles, oleadas, balances
levels/	
level_01.json	

6.4. Diseño de Clases

Propone un esquema polimórfico:

Game y State: Coordinan el flujo entre los menús y la partida principal.

Enemy y Defense: Subclases especializadas (ej. GoblinSaqueador, TorreArquera) que interaccionan con una clase Grid del mapa.

Managers: Actúan como controladores lógicos explícitos (ej. CollisionManager, ResourceManager) comunicándose a través de las referencias del estado activo.

6.5. Ventajas Observadas

Facilidad de entendimiento OOP: Utiliza un diseño polimórfico limpio que facilita implementar nuevas torres y estados sin complejidad abstracta.

Recomendaciones ingenieriles muy directas: Aconseja control de versiones con Git (commits atómicos), docstrings para documentación en el código, y separación de la presentación de la lógica de negocio.

6.6. Desventajas Observadas

Su arquitectura acopla módulos un poco más que Claude o Gemini debido a que utiliza referencias explícitas hacia los Managers en vez de un bus de eventos (aunque reconoce un Event Bus como una posible mejora futura).

7. Respuesta de Claude

7.1. Descripción General

Claude presenta una arquitectura en capas combinada con el patrón Game Loop y un sistema Entity-Component-System (ECS) simplificado. Su enfoque detalla minuciosamente el desacoplamiento de componentes.

7.2. Arquitectura Propuesta

Estructura el proyecto en tres niveles:

Capa de presentación: Pygame (renderizado e inputs).

Capa lógica: Sistemas de juego controlados por eventos (Event Bus).

Capa de datos: Configuraciones y definiciones separadas del código.

7.3. Organización de Carpetas

el_ultimo_bastion/

```
|
|
| — src/                # Código fuente principal
| | — core/             # Núcleo del motor
| | | — game.py         # Game loop principal, inicialización
| | | — state_machine.py # Máquina de estados (pantallas)
| | | — event_bus.py    # Sistema de eventos (Observer)
| | | — settings.py     # Constantes globales y config runtime
| | | — asset_loader.py  # Carga centralizada de recursos
| |
| | — entities/         # Entidades del juego
```

```

| | | └─ base_entity.py      # Clase base abstracta
| | | └─ enemies/
| | |   └─ base_enemy.py
| | |   └─ goblin_raider.py
| | |   └─ goblin_archer.py
| | |   └─ goblin_brute.py
| | | └─ defenses/
| | |   └─ base_defense.py
| | |   └─ archer_tower.py
| | |   └─ defensive_wall.py
| | |   └─ ballista.py
| | └─ bastion.py           # El bastión (entidad especial)
| |
| └─ systems/              # Sistemas de lógica
| | └─ wave_system.py      # Control de oleadas
| | └─ combat_system.py    # Resolución de combate / colisiones
| | └─ resource_system.py  # Gestión de oro y maná
| | └─ build_system.py     # Construcción y colocación
| | └─ pathfinding.py      # Navegación de enemigos (A*)
| | └─ map_system.py       # Grid del mapa, celdas válidas
| |
| └─ screens/              # Pantallas (estados)
| | └─ main_menu.py
| | └─ gameplay.py
| | └─ pause_screen.py
| | └─ defeat_screen.py
  
```

```

| |
| |— ui/          # Interfaz de usuario
| | |— hud.py      # HUD en gameplay (recursos, vida, oleada)
| | |— build_panel.py  # Panel de construcción
| | |— tooltip.py   # Tooltips de estructuras
| | |— button.py    # Componente botón reutilizable
| |
| |— utils/        # Utilidades transversales
| | |— timer.py     # Temporizadores reutilizables
| | |— animation.py # Gestión de sprites animados
| | |— math_utils.py # Distancias, vectores 2D
|
|— assets/         # Recursos estáticos
| |— sprites/
| | |— enemies/
| | |— towers/
| | |— ui/
| | |— map/
| | |— effects/
| | |— sounds/
| | |— sfx/
| | |— music/
| |— fonts/
|
|— data/           # Configuración en datos (no código)
| |— enemies.json  # Stats de todos los enemigos
  
```

```

|   |— towers.json          # Costos y stats de torres
|   |— waves.json          # Definición de cada oleada
|   |— maps/
|       |— map_01.json      # Layout del mapa
|
|— tests/                  # Pruebas unitarias
|   |— test_wave_system.py
|   |— test_combat_system.py
|   |— test_resource_system.py
|   |— test_pathfinding.py
|
|— docs/                  # Documentación
|   |— architecture.md
|   |— game_design.md
|   |— api_reference.md
|
|— main.py                # Punto de entrada
|— requirements.txt
|— README.md

```

7.4. Diseño de Clases

BaseEntity: Clase raíz abstracta enfocada al uso de delta time (dt).

BaseEnemy y BaseDefense: Implementan herencia gestionando daño, rangos y recompensas.

WaveSystem: Encargado de leer archivos JSON para crear escalado de dificultad.

EventBus: Clase Singleton vital para publicar y suscribir eventos, clave del diseño de Claude.

7.5. Ventajas Observadas

Uso estricto del Delta Time y Event Bus: Impide dependencias circulares y estandariza velocidades en múltiples tasas de refresco (FPS).

Prácticas eficientes en Pygame: Propone usar pygame.sprite.Group y precarga de caché para mitigar cuellos de botella de memoria.

Orden de desarrollo estructurado: Brinda 8 pasos exactos para generar un producto jugable desde la primera semana.

7.6. Desventajas Observadas

Su enfoque orientado a eventos y ECS requiere disciplina desde el inicio (EventBus es "no-negociable"), lo que puede aumentar la complejidad técnica inicial para un desarrollador inexperto.

Requiere precalcular las rutas (pathfinding) debido al alto impacto que tendría en CPU.

8. Comparación Técnica Inicial

8.1. Criterios de Evaluación

Para la comparación se definieron los siguientes criterios:

- Organización del proyecto.
- Modularidad.
- Escalabilidad.
- Claridad de explicación.
- Diseño orientado a objetos.
- Facilidad de implementación.
- Calidad general.

8.2. Escala de Valoración

Valor	Interpretación
1	Muy deficiente

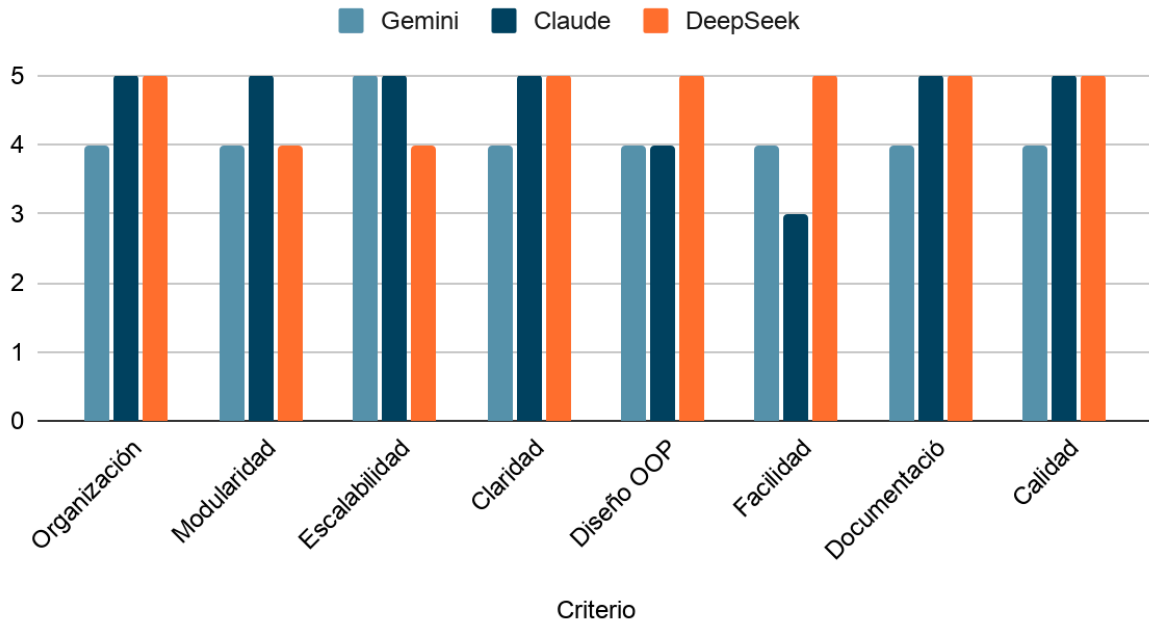
2	Deficiente
3	Aceptable
4	Bueno
5	Excelente

8.3. Tabla Comparativa

Criterio	Gemini	Claude	DeepSeek
Organización	4	5	5
Modularidad	4	5	4
Escalabilidad	5	5	4
Claridad	4	5	5
Diseño OOP	4	4	5
Facilidad de Implementación	4	3	5
Documentación	4	5	5
Calidad General	4	5	5

8.4. Gráfico Comparativo

Gemini, Claude y DeepSeek



9. Análisis de Resultados

9.1. Organización

Claude y DeepSeek destacaron al proveer estructuras de directorios sumamente exhaustivas e incluir la justificación de sus decisiones archivo por archivo. Claude resaltó particularmente por su estricta separación de los datos de configuración (como archivos JSON para estadísticas) fuera del árbol de código fuente. La estructura de Gemini, aunque adecuada y profesional, fue ligeramente menos detallada que las otras dos propuestas.

9.2. Escalabilidad

Claude presentó la arquitectura más escalable al imponer un diseño basado en el patrón Event Bus (Observer) como regla no negociable, lo que permite agregar nuevos enemigos, torres y mecánicas sin alterar las clases ya existentes ni provocar dependencias. Gemini también destacó proponiendo un diseño Data-Driven y el patrón Factory para escalar el proyecto. DeepSeek ofrece escalabilidad tradicional a través de polimorfismo, aunque admite

que a gran escala podría haber problemas de acoplamiento al usar referencias directas en lugar de eventos.

9.3. Claridad

Claude demostró una claridad superior al explicar los flujos del sistema con diagramas de dependencias implícitos en su texto, el uso del motor Pygame, y ofrecer un paso-a-paso para programar el proyecto semana a semana. DeepSeek también presentó una claridad sobresaliente detallando metódicamente cada atributo y método principal de sus clases. Gemini entregó tablas muy legibles, pero su desglose arquitectónico fue menos visual.

9.4. Calidad General

Tanto Claude como DeepSeek generaron respuestas comparables a las de un Arquitecto de Software Senior. Claude sobresalió por su énfasis en la arquitectura de eventos y mitigación rigurosa de ineficiencias de Python, mientras que DeepSeek brilló en los principios clásicos de la POO y las sugerencias de control de calidad del código (docstrings y pytest). La respuesta de Gemini también es funcional y de alta calidad para el proyecto, pero menos profunda técnicamente en la justificación de las mitigaciones.

9.5. Observaciones

Un patrón notable entre las tres IA es su insistencia en externalizar todos los parámetros de balanceo y estadísticas hacía archivos .json (Data-Driven Design) para facilitar futuras iteraciones. Asimismo, todas advirtieron sobre los cuellos de botella del rendimiento inherentes a Pygame (procesamiento en CPU y cálculos de colisiones), sugiriendo medidas unánimes como el Object Pooling y el control por Delta Time.

10. Discusión

Los resultados obtenidos evidencian que las inteligencias artificiales pueden contribuir significativamente en las primeras etapas del desarrollo de software, especialmente en actividades relacionadas con análisis, planificación y diseño arquitectónico. Sin embargo, las

diferencias observadas entre las respuestas permiten identificar fortalezas particulares de cada herramienta, las cuales deberán ser consideradas en las siguientes etapas del proyecto. La comparación realizada constituye una primera aproximación para evaluar la utilidad práctica de estas tecnologías dentro de proyectos de ingeniería de software.

11. Resultados Obtenidos

Al finalizar esta etapa se logró:

- Diseñar un Prompt Maestro común.
- Obtener propuestas de tres IA distintas.
- Analizar arquitecturas de software.
- Realizar una comparación inicial.
- Identificar fortalezas y debilidades de cada IA.

12. Conclusiones

Las inteligencias artificiales son capaces de generar propuestas de arquitectura útiles, profundas y aplicables para proyectos de desarrollo de software como "El Último Bastión".

El uso de un Prompt Maestro permitió mantener condiciones homogéneas para la evaluación, comprobando que las diferencias emanan de la capacidad propia de los motores de Gemini, Claude y DeepSeek.

Las respuestas obtenidas mostraron diferencias en organización, claridad y nivel de detalle: Claude sobresaliendo en el bajo acoplamiento por eventos, DeepSeek en la limpieza de la Programación Orientada a Objetos, y Gemini en la rápida implementación modular. La comparación realizada servirá como base para las etapas posteriores de implementación y evaluación práctica.

El análisis inicial evidencia el enorme potencial de las IA como herramientas de apoyo técnico, planeación de mitigación de riesgos (como la gestión de recursos de Pygame) y establecimiento de buenas prácticas para desarrolladores de software.

Anexo

Link de Recursos y Proyecto

https://github.com/mariovicunaa/Proyecto_El_Ultimo_Bastion

Referencias

Mejía Trejo, J. (2024). *Principios de aseguramiento de calidad para el diseño de software: innovación de procesos en las tecnologías de información*: (1 ed.). Academia Mexicana de Investigación y Docencia en Innovación (AMIDI).

<https://elibro.net/es/lc/uguayaquil/titulos/250101>

Schwaber, K., & Sutherland, J. (2020, 11). *La Guía Scrum*. Scrumguides. Retrieved 06 11, 2026, from

<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>

Winters, T. & Manshreck, T. (2022). *Ingeniería de software en Google: Lecciones sobre programación aprendidas a lo largo del tiempo*: (1 ed.). Marcombo.

<https://elibro.net/es/lc/uguayaquil/titulos/281863>